

Reviewer Pack — Euler Characteristic of Communication Graph Equals Determini...

Entry 1d0355b7e76a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 19:26:42 UTC

Euler Characteristic of Communication Graph Equals Deterministic Communication Complexity

Entry ID: 1d0355b7e76a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 19:26:42 UTC

1. Conjecture

Field A (mathematical branch): Algebraic topology (Euler characteristic of simplicial complexes)

Field B (complexity object): Deterministic communication complexity of Boolean functions

Statement:

For any Boolean function f , the Euler characteristic of the clique complex of its communication graph is equal to the deterministic communication complexity of f .

Rationale (proposer's reasoning):

The Euler characteristic captures topological invariants that may reflect the structural complexity of the communication graph, which could directly relate to the required communication. This bridge might expose how geometric features of the interaction graph correlate with communication cost.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d9995c2825cde2fe

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Euler characteristic communication complexity Boolean function - simplicial complex Euler characteristic deterministic communication complexity - clique complex Euler characteristic deterministic communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/1511.07912v3] The Thom-Sebastiani theorem for the Euler characteristic of cyclic L-infinity algebras - [http://arxiv.org/abs/2509.13173v1] Euler's explorations of extremal ellipses - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1112.4523v1] Complexity and Algorithms for Euler Characteristic of Simplicial Complexes - [http://arxiv.org/abs/1808.02841v1] On divergent Series - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=-9, elapsed=95.4s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def matrix_multiply(A, B):
    n = len(A)
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
```

```

        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0]*n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    for c in range(n):
        submatrix = [row[:c] + row[c+1:] for row in A[1:]]
        det += ((-1)**c) * A[0][c] * determinant(submatrix)
    return det

def euler_characteristic(clique_complex):
    n = len(clique_complex)
    return sum((-1)**k * len(clique_complex[k]) for k in range(n))

def communication_graph(f, n):
    vertices = [(i, j) for i in range(2**n) for j in range(i+1, 2**n)]
    edges = []
    for u, v in vertices:
        if f(u) != f(v):
            edges.append((u, v))
    return vertices, edges

def deterministic_communication_complexity(f, n):
    # Placeholder for actual computation
    return 0 # Replace with actual implementation

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    f = lambda x: sum(x[i] for i in range(n)) % 2 # Example Boolean function
    vertices, edges = communication_graph(f, n)
    clique_complex = [[] for _ in range(n+1)]
    for edge in edges:
        u, v = edge
        for k in range(n):
            if (u & (1 << k)) != (v & (1 << k)):
                break
        else:
            continue
        clique_complex[k].append(edge)
    euler_char = euler_characteristic(clique_complex)
    comm_complexity = deterministic_communication_complexity(f, n)
    diff = abs(euler_char - comm_complexity)
    return {
        "metric_name": "Absolute Difference",
        "metric_value": diff,
    }

```

```

        "instances_tested": 1,
        "conjecture_holds": diff == 0,
        "counterexample": "" if diff == 0 else f"Diff: {diff}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\"}, \"metric_value\": {result['metric_value']}}")
        results.append(result)

    mean_diff = sum(r['metric_value'] for r in results) / len(results)
    std_dev = math.sqrt(sum((r['metric_value'] - mean_diff)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if not r['conjecture_holds']) / len(results)

    if support_fraction < 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={results[0]['seed']}")
    else:
        print(f"RESULT: SUPPORTED mean={mean_diff} std={std_dev} support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing data, preventing evaluation of support fraction or counterexamples. | next: Re-run test with enhanced logging to diagnose crash cause and validate conjecture

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	100029
2	preregistration	ollama_remote	qwen3:8b	0	0	24040
3	novelty	ollama_remote	qwen3:8b	0	0	21955

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	qwen3:8b	0	0	18059
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18448
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31359
7	judge	ollama_remote	qwen3:8b	0	0	126883

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 340773 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/1d0355b7e76a.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/1d0355b7e76a.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1d0355b7e76a.tar.gz (if generated)